Universidade Federal da Paraíba
Centro de Informática
Disciplina: Engenharia de Sistemas Distribuídos

Proposta de Projeto Final: TicketFlow

- **Integrantes:**
- Herick de Lima Carneiro (*herickcarneiro@gmail.com*)
- João Lucas Félix de Macena (*jlmacena06@gmail.com*)
- João Victor Lourenço da Silva (*joao.lourenco@academico.ufpb.br*)
- Luana de Méro Omena (*luanamero4@gmail.com*)
- Luerson de Albuquerque Silva Filho (*luerson.filho@academico.ufpb.br*)
- Leandeson Pinheiro Santos de Araújo (leandesonpinheiro20@gmail.com)

- **Tema escolhido e proposta:**

O tema escolhido pela equipe foi um dos disponibilizados pelo professor no documento que descreve o projeto final:

> Venda de ingressos com reserva de assentos: reserva temporária, pagamento simulado, idempotência em tentativas repetidas, SAGA para compensação e Transactional Outbox.

A partir desse tema, a equipe propõe o projeto **TicketFlow: Plataforma Distribuída de Venda de Ingressos com Reserva de Assentos**. O problema a ser resolvido é o da abertura de vendas de um evento, momento em que milhares de usuários disputam os mesmos assentos ao mesmo tempo. Sem o devido controle, o sistema pode vender um mesmo assento mais de uma vez, cobrar o cliente em dobro quando ele repete o clique ou a rede falha, e deixar dados inconsistentes caso algum serviço caia no meio do fluxo de reserva, pagamento e emissão do ingresso.

Para isso, a solução será construída em uma arquitetura de microsserviços, inicialmente composta pelos seguintes serviços:

- API Gateway (ponto único de entrada das requisições);
- Serviço de Eventos e Assentos (catálogo de eventos e disponibilidade dos assentos);
- Serviço de Reservas (reserva temporária do assento, com expiração automática);
- Serviço de Pagamentos (pagamento simulado, com tratamento de idempotência);
- Serviço de Notificações (envio assíncrono da confirmação e do ingresso).

Diante do tema selecionado, em discussão realizada de forma remota, a equipe elegeu 2 áreas técnicas como essenciais para o tipo de projeto que o tema direciona: Segurança e Confiabilidade.

Com as áreas técnicas definidas, apesar de tratar-se apenas de uma ideia inicial para o projeto, foram definidos 4 padrões que, em conjunto, foram julgados como adequados para aplicação:

- API e API Gateway Pattern (para auxílio no gerenciamento da distribuição das requisições com autenticação, autorização, rate limiting);
- SAGA Pattern (para coordenar o fluxo de reserva, pagamento e emissão, liberando o assento como compensação em caso de falha);
- Transactional Outbox Pattern (para garantir que os eventos só sejam publicados quando a transação no banco for confirmada);
- Circuit Breaker (para isolar falhas do serviço de pagamento e evitar que se propaguem pelo sistema);

Além dos padrões, todas as operações de escrita utilizarão chave de idempotência, de forma a tolerar tentativas repetidas sem efeitos duplicados.

- **Tecnologias escolhidas:**

Spring framework com algumas de suas extensões: Spring Boot para facilitar o processo de desenvolvimento e Spring Cloud para auxiliar na arquitetura de microsserviços, com destaque para o Spring Cloud Gateway na implementação do API Gateway e o Resilience4j para o Circuit Breaker. Como complemento, estão previstos o PostgreSQL como banco de dados de cada serviço, o Redis para a reserva temporária dos assentos e o rate limiting, o Kafka ou RabbitMQ como broker de mensagens, o Keycloak para autenticação com OAuth 2.0 e JWT, e o Docker Compose com GitHub Actions para execução local e integração contínua.

- **Critérios de sucesso:**

Para considerar o projeto bem-sucedido, a equipe definiu os seguintes critérios mensuráveis, que poderão ser ajustados ao longo do desenvolvimento:

- Nenhum assento vendido em duplicidade com 500 usuários simultâneos disputando os mesmos assentos;
- Nenhuma cobrança duplicada em requisições repetidas com a mesma chave de idempotência;
- Liberação automática de reservas não pagas em até 10 minutos;
- Latência p95 da reserva abaixo de 300 ms com carga de 200 requisições por segundo;
- Resposta de erro controlada em menos de 1 segundo quando o serviço de pagamento estiver indisponível;
- Nenhum evento perdido ao derrubar um serviço durante a publicação de mensagens;

- Compensação (liberação do assento) em 100% das reservas cujo pagamento falhar.
- **Plano de Testes:**

Os testes foram pensados para validar diretamente as decisões arquiteturais e os critérios de sucesso definidos acima:

- Testes de concorrência, enviando centenas de requisições simultâneas para o mesmo assento e verificando que apenas uma reserva é confirmada;
- Testes de idempotência, reenviando a mesma requisição de pagamento e verificando que apenas uma cobrança é registrada;
- Testes de carga com k6, simulando o pico de abertura de vendas com carga progressiva e medindo latência (p95) e taxa de erro;
- Testes de resiliência, derrubando o serviço de pagamento e o broker de mensagens para verificar o funcionamento do Circuit Breaker, da compensação da SAGA e da entrega posterior dos eventos do Outbox;
- Testes de integração com Testcontainers, executando o fluxo completo de reserva, pagamento e emissão com banco de dados e broker reais.
- **Repositório:**

O repositório Git do projeto, com o README inicial, está disponível em: **https://github.com/jovilourenco/projeto-eng_sistemas_distribuidos**